```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <math.h>
```

```
// ================ WIFI SETTINGS ================
const char* ssid = "iPhone";
const char* password = "1234567890";
```

```
// ================ MPU6050 ================
#define MPU_SDA 18
#define MPU_SCL 19
#define MPU_ADDR 0x68
```

```
TwoWire MPUWire(0);
```

```
// ================ OLED SSD1306 ================
#define OLED_SDA 21
#define OLED_SCL 22
#define OLED_ADDR 0x3C
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
```

```
TwoWire OLEDWire(1);
Adafruit_SSD1306 display(
  SCREEN_WIDTH, SCREEN_HEIGHT, &OLEDWire, -1
);
```

```
// ================ FSR SENSOR ================
#define FSR_PIN 34
```

```
// ================ DS18B20 ================
#define TEMP_PIN 4
```

```
OneWire oneWire(TEMP_PIN);
DallasTemperature tempSensor(&oneWire);
```

```
// ================ WEB SERVER ================
WebServer server(80);
```

```
// ================ SENSOR DATA ================
unsigned long steps = 0;
int fsrValue = 0;
float temperatureC = NAN;
float accelerationG = 1.0;
```

```
bool mpuOK = false;
bool oledOK = false;
bool tempOK = false;

unsigned long lastStepTime = 0;
unsigned long lastSensorTime = 0;
unsigned long lastSerialTime = 0;
unsigned long lastTempRequest = 0;

bool stepArmed = true;
bool tempConversionPending = false;

const float STEP_THRESHOLD = 1.18;
const unsigned long STEP_DELAY = 300;

// ================ MPU6050 FUNCTIONS ================
bool writeMPU(byte reg, byte value) {
  MPUWire.beginTransmission(MPU_ADDR);
  MPUWire.write(reg);
  MPUWire.write(value);
  return MPUWire.endTransmission() == 0;
}

bool readMPUData(int16_t &ax, int16_t &ay, int16_t &az) {
  MPUWire.beginTransmission(MPU_ADDR);
  MPUWire.write(0x3B);

  if (MPUWire.endTransmission(false) != 0) {
    return false;
  }

  if (MPUWire.requestFrom(MPU_ADDR, 6, true) != 6) {
    return false;
  }

  ax = (int16_t)((MPUWire.read() << 8) | MPUWire.read());
  ay = (int16_t)((MPUWire.read() << 8) | MPUWire.read());
  az = (int16_t)((MPUWire.read() << 8) | MPUWire.read());

  return true;
}

// ================ STEP DETECTION ================
void updateSteps() {
  int16_t ax, ay, az;

  if (!mpuOK || !readMPUData(ax, ay, az)) {
    return;
  }
```

```
  // MPU6050 accelerometer range: +/-2g
  float x = ax / 16384.0f;
  float y = ay / 16384.0f;
  float z = az / 16384.0f;

  accelerationG = sqrtf(x * x + y * y + z * z);

  unsigned long now = millis();

  // Basic prototype peak detection
  if (accelerationG < 1.08f) {
    stepArmed = true;
  }

  if (stepArmed &&
      accelerationG > STEP_THRESHOLD &&
      now - lastStepTime >= STEP_DELAY) {
    steps++;
    lastStepTime = now;
    stepArmed = false;
  }
}

// ================ SENSOR UPDATE ================
void updateSensors() {
  fsrValue = analogRead(FSR_PIN);
  updateSteps();

  // Read completed DS18B20 conversion
  if (tempConversionPending &&
      millis() - lastTempRequest >= 800) {

    float t = tempSensor.getTempCByIndex(0);

    if (t != DEVICE_DISCONNECTED_C &&
        t >= -55.0f && t <= 125.0f) {
      temperatureC = t;
      tempOK = true;
    } else {
      temperatureC = NAN;
      tempOK = false;
    }

    tempConversionPending = false;
  }

  // Start the next temperature conversion
  if (!tempConversionPending &&
      millis() - lastTempRequest >= 1000) {
```

```
    tempSensor.requestTemperatures();
    lastTempRequest = millis();
    tempConversionPending = true;
  }
}

// ================ OLED DISPLAY ================
void updateOLED() {
  if (!oledOK) return;

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("LEGCARE MONITOR");
  display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

  display.setCursor(0, 14);
  display.print("Steps: ");
  display.println(steps);

  display.setCursor(0, 25);
  display.print("FSR: ");
  display.println(fsrValue);

  display.setCursor(0, 36);
  display.print("Temp: ");

  if (tempOK) {
    display.print(temperatureC, 1);
    display.println(" C");
  } else {
    display.println("-- C");
  }

  display.setCursor(0, 48);
  display.print("WiFi: ");
  display.println(WiFi.status() == WL_CONNECTED
                  ? "Connected" : "Offline");

  display.display();
}

// ================ WEB DASHBOARD HTML ================
const char webpage[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width,initial-scale=1">
<meta name="theme-color" content="#09111f">
<title>LEGCARE | Biomedical Monitor</title>
```

```
<style>
:root {
  --bg:#09111f;
  --card:#121e30;
  --line:#26364c;
  --text:#f1f5f9;
  --muted:#9eafc5;
  --mint:#65e0c0;
  --blue:#8bb8ff;
  --orange:#ffc078;
}
* { box-sizing:border-box; }
body {
  margin:0;
  background:var(--bg);
  color:var(--text);
  font-family:Arial,Helvetica,sans-serif;
}
main {
  max-width:1000px;
  margin:auto;
  padding:24px 16px 40px;
}
header {
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:12px;
  margin-bottom:25px;
}
.brand {
  display:flex;
  gap:12px;
  align-items:center;
}
.logo {
  width:46px;
  height:46px;
  border-radius:14px;
  display:grid;
  place-items:center;
  background:#193b43;
  color:var(--mint);
  font-size:26px;
}
```

```
h1 {
  margin:0;
  font-size:22px;
  letter-spacing:1px;
}
.subtitle {
  color:var(--muted);
  font-size:11px;
  margin-top:5px;
}
.status {
  color:var(--orange);
  border:1px solid #6a5230;
  background:#332819;
  padding:8px 11px;
  border-radius:30px;
  font-size:11px;
  white-space:nowrap;
}
.status.online {
  color:var(--mint);
  border-color:#276457;
  background:#12342f;
}
.intro { margin:22px 0; }
.intro h2 { font-size:27px; margin:0 0 8px; }
.intro p { color:var(--muted); margin:0; font-size:14px; }
.grid {
  display:grid;
  grid-template-columns:repeat(2,minmax(0,1fr));
  gap:14px;
}
.card {
  background:var(--card);
  border:1px solid var(--line);
  border-radius:18px;
  padding:20px;
  min-width:0;
}
.card-top {
  display:flex;
  align-items:center;
  justify-content:space-between;
  gap:8px;
}
.label { color:var(--muted); font-size:13px; }
.symbol { font-size:23px; }
.value {
```

```
  font-size:clamp(27px,5vw,38px);
  font-weight:700;
  margin:20px 0 8px;
  overflow-wrap:anywhere;
}
.unit { font-size:15px; color:var(--muted); }
.note { font-size:12px; color:var(--muted); }
.mint { color:var(--mint); }
.blue { color:var(--blue); }
.orange { color:var(--orange); }
.section-title { margin:28px 0 12px; font-size:16px; }
.panel {
  background:var(--card);
  border:1px solid var(--line);
  border-radius:18px;
  padding:18px;
}
.bar {
  height:9px;
  background:#29384c;
  border-radius:20px;
  overflow:hidden;
  margin:15px 0 9px;
}
.fill {
  height:100%;
  width:0%;
  border-radius:20px;
  background:var(--mint);
  transition:width .3s;
}
.details {
  display:flex;
  justify-content:space-between;
  gap:12px;
  font-size:12px;
  color:var(--muted);
}
.row {
  display:flex;
  justify-content:space-between;
  gap:12px;
  padding:13px 0;
  border-bottom:1px solid var(--line);
  font-size:13px;
}
.row span:first-child { color:var(--muted); }
.row:last-child { border-bottom:0; }
```

```
footer {
  color:var(--muted);
  font-size:11px;
  line-height:1.7;
  margin-top:22px;
}
@media(max-width:520px) {
  main { padding:18px 12px 30px; }
  .grid { gap:10px; }
  .card { padding:14px; border-radius:15px; }
  h1 { font-size:19px; }
  .intro h2 { font-size:23px; }
  .subtitle { font-size:9px; }
}
</style>
</head>
```

```
<body>
<main>
<header>
  <div class="brand">
    <div class="logo">~</div>
    <div>
      <h1>LEGCARE</h1>
      <div class="subtitle">BIOMEDICAL MONITORING SYSTEM</div>
    </div>
  </div>
  <div id="status" class="status">● CONNECTING</div>
</header>
```

```
<section class="intro">
  <h2>Monitoring Dashboard</h2>
  <p>Live sensor readings from your ESP32 device.</p>
</section>
```

```
<section class="grid">
  <div class="card">
    <div class="card-top">
      <span class="label">Step Count</span>
      <span class="symbol mint">↗</span>
    </div>
    <div class="value" id="steps">--</div>
    <div class="note">Detected movement steps</div>
  </div>
```

```
  <div class="card">
    <div class="card-top">
      <span class="label">FSR Pressure</span>
      <span class="symbol blue">◉</span>
```

```
    </div>
    <div class="value" id="fsr">--</div>
    <div class="note">Raw ADC reading · 0–4095</div>
  </div>
```

```
  <div class="card">
    <div class="card-top">
      <span class="label">Temperature</span>
      <span class="symbol orange">🌡</span>
    </div>
    <div class="value"><span id="temp">--</span>
      <span class="unit">°C</span></div>
    <div class="note">DS18B20 reading</div>
  </div>
```

```
  <div class="card">
    <div class="card-top">
      <span class="label">Acceleration</span>
      <span class="symbol mint">〰</span>
    </div>
    <div class="value"><span id="accel">--</span>
      <span class="unit">g</span></div>
    <div class="note">MPU6050 magnitude</div>
  </div>
</section>
```

```
<h3 class="section-title">Pressure Sensor Level</h3>
<div class="panel">
  <div class="details">
    <span>FSR ADC level</span>
    <strong id="pressureLabel">Waiting for data</strong>
  </div>
  <div class="bar"><div class="fill" id="pressureFill"></div></div>
  <div class="details">
    <span>0</span><span>2048</span><span>4095</span>
  </div>
</div>
```

```
<h3 class="section-title">Device Information</h3>
<div class="panel">
   <div class="row"><span>Wi-Fi</span><strong id="wifi">--</strong></div>
  <div class="row"><span>MPU6050</span><strong id="mpu">--
</strong></div>
  <div class="row"><span>DS18B20</span><strong id="tempStatus">--
</strong></div>
  <div class="row"><span>Last update</span><strong id="updated">--
</strong></div>
</div>
```

```
<footer>
  Educational prototype only. FSR values are raw ADC readings, not
calibrated
  force measurements. Step detection requires testing and tuning.
Temperature
  and movement readings are not a medical diagnosis.
</footer>
</main>
```

```
<script>
async function refreshData() {
  try {
    const response = await fetch('/data', {cache:'no-store'});
    if (!response.ok) throw new Error('Request failed');
    const d = await response.json();
```

```
    document.getElementById('steps').textContent = d.steps;
    document.getElementById('fsr').textContent = d.fsr;
    document.getElementById('accel').textContent =
      Number(d.accel).toFixed(2);
    document.getElementById('temp').textContent =
      d.tempOK ? Number(d.temp).toFixed(1) : '--';
```

```
    const pct = Math.max(0,Math.min(100,d.fsr/4095*100));
    document.getElementById('pressureFill').style.width = pct+'%';
```

```
    let level = 'LOW';
    if (d.fsr >= 2500) level = 'HIGH';
    else if (d.fsr >= 1200) level = 'MODERATE';
```

```
    document.getElementById('pressureLabel').textContent =
      level+' · '+pct.toFixed(0)+'%';
```

```
    document.getElementById('wifi').textContent =
      d.wifi ? 'Connected' : 'Disconnected';
    document.getElementById('mpu').textContent =
      d.mpuOK ? 'Detected' : 'Not detected';
    document.getElementById('tempStatus').textContent =
      d.tempOK ? 'Detected' : 'Unavailable';
    document.getElementById('updated').textContent =
      new Date().toLocaleTimeString();
```

```
    const status = document.getElementById('status');
    status.textContent = '● LIVE';
    status.classList.add('online');
  } catch (e) {
    const status = document.getElementById('status');
    status.textContent = '● RECONNECTING';
    status.classList.remove('online');
```

```
  }
}
```

```
refreshData();
setInterval(refreshData,1000);
</script>
</body>
</html>
)rawliteral";
```

```
// ================ WEB ROUTES ================
void handleRoot() {
  server.send_P(200, "text/html", webpage);
}
```

```
void handleData() {
  String json = "{";
  json += "\"steps\":" + String(steps) + ",";
  json += "\"fsr\":" + String(fsrValue) + ",";
  json += "\"accel\":" + String(accelerationG, 3) + ",";
  json += "\"temp\":" + String(tempOK ? temperatureC : 0.0f, 2) + ",";
  json += "\"tempOK\":" + String(tempOK ? "true" : "false") + ",";
  json += "\"mpuOK\":" + String(mpuOK ? "true" : "false") + ",";
  json += "\"wifi\":" + String(
    WiFi.status() == WL_CONNECTED ? "true" : "false"
  );
  json += "}";
```

```
  server.send(200, "application/json", json);
}
```

```
// ================ SETUP ================
void setup() {
  Serial.begin(115200);
  delay(500);
```

```
  analogReadResolution(12);
  pinMode(FSR_PIN, INPUT);
```

```
  // Start MPU6050 I2C bus
  MPUWire.begin(MPU_SDA, MPU_SCL, 100000);
```

```
  MPUWire.beginTransmission(MPU_ADDR);
  mpuOK = (MPUWire.endTransmission() == 0);
```

```
  if (mpuOK) {
    writeMPU(0x6B, 0x00); // Wake up
    writeMPU(0x1C, 0x00); // +/-2g
    Serial.println("MPU6050 detected");
  } else {
```

```
    Serial.println("MPU6050 not detected; check SDA/SCL.");
  }
```

```
  // Start OLED on its own I2C bus
  OLEDWire.begin(OLED_SDA, OLED_SCL, 100000);
```

```
  oledOK = display.begin(
    SSD1306_SWITCHCAPVCC, OLED_ADDR, false, false
  );
```

```
  if (oledOK) {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    display.setTextSize(1);
    display.setCursor(0, 0);
    display.println("LEGCARE");
    display.println("Starting...");
    display.display();
  } else {
    Serial.println("OLED not detected; check wiring/address.");
  }
```

```
  // DS18B20 setup
  tempSensor.begin();
  tempSensor.setResolution(12);
  tempSensor.setWaitForConversion(false);
  tempSensor.requestTemperatures();
  lastTempRequest = millis();
  tempConversionPending = true;
```

```
  // Connect to PROJECT Wi-Fi
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
```

```
  Serial.print("Connecting to PROJECT Wi-Fi");
```

```
  unsigned long startAttempt = millis();
```

```
  while (WiFi.status() != WL_CONNECTED &&
         millis() - startAttempt < 20000) {
    delay(500);
    Serial.print(".");
  }
```

```
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nWi-Fi connected!");
    Serial.print("Dashboard URL: http://");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\nWi-Fi connection failed.");
```

```
    Serial.println("Check SSID/password and 2.4GHz hotspot.");
  }

  server.on("/", HTTP_GET, handleRoot);
  server.on("/data", HTTP_GET, handleData);
  server.begin();

  Serial.println("Web server started");
}

// ================ MAIN LOOP ================
void loop() {
  server.handleClient();

  if (millis() - lastSensorTime >= 50) {
    lastSensorTime = millis();

    updateSensors();
    updateOLED();

    if (millis() - lastSerialTime >= 1000) {
      lastSerialTime = millis();

      Serial.print("Steps: ");
      Serial.print(steps);

      Serial.print(" | FSR: ");
      Serial.print(fsrValue);

      Serial.print(" | Temperature: ");
      if (tempOK) Serial.print(temperatureC, 1);
      else Serial.print("N/A");

      Serial.print(" C | Acceleration: ");
      Serial.print(accelerationG, 2);
      Serial.println(" g");
    }
  }
}
```

1. Complete connections

Component pin	ESP32 pin
MPU6050 VCC	3.3V
MPU6050 GND	GND
MPU6050 SDA	GPIO 18
MPU6050 SCL	GPIO 19
MPU6050 AD0	GND
OLED VCC	3.3V
OLED GND	GND
OLED SDA	GPIO 21
OLED SCL	GPIO 22
FSR Pin 1	3.3V
FSR Pin 2	GPIO 34
FSR 10 kΩ resistor	GPIO 34 to GND
DS18B20 VDD	3.3V
DS18B20 DATA	GPIO 4
DS18B20 GND	GND
DS18B20 4.7 kΩ resistor	DATA to 3.3V